EXPERIMENT 14

ExploringPro-CodeExtension of a Zoho Creator Application Using Zoho Catalyst and Custom Functions

Patient AppointmentSchedulingSystem: PriorityScoringExtensionUseCase

Sreeman Naibal S
241401106

1. Aim

To explore how an appointment scheduling application built in Zoho Creator can be extended beyond low-code by using a reusable Deluge custom function and a pro-code serverless function on Zoho Catalyst, so that logic too involved for simple workflow actions can still be added.

An appointment priority scoring example is used to show how the two parts connect, when each option is appropriate, and which precautions a real implementation needs.

2. Procedure

Step 1: Find the extension need. The patient appointment form and its workflow in the Zoho Creator application were reviewed, and a requirement was chosen that built-in actions handle poorly: giving each appointment a Normal, Medium or High priority from the patient's age and existing conditions.

Step 2: Compare the options. Workflow actions, Deluge custom functions, a Creator Custom API and a Catalyst function were compared on skill needed and typical use (Section 3.2).

Step 3: Design the architecture. The Creator side (form, workflow, custom function, connection) and the Catalyst side (Advanced I/O function with optional storage) were drawn in one diagram (Section 3.1).

Step 4: Draft the Deluge function. A reusable standalone function was drafted that builds a JSON payload and calls the Catalyst endpoint through a connection with invokeurl (Section 3.3).

Step 5: Draft the Catalyst function. A Node.js Advanced I/O function was drafted that checks the input, applies the scoring rules and returns the priority as JSON (Section 3.4).

Step 6: Plan the flow and the choice. The sequence from form submission to record update, including the failure path, was drawn as a flowchart (Section 3.5), and a guide for choosing the extension type was prepared (Section 3.6).

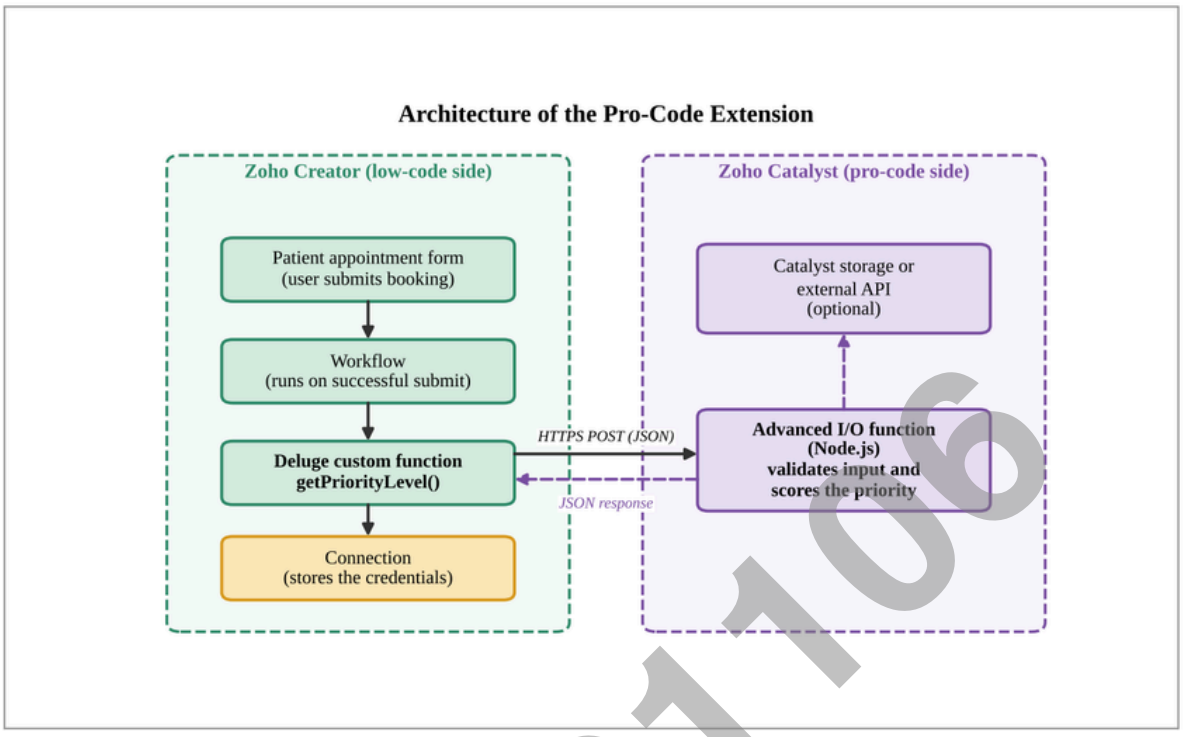
Step7:Listtheprecautions. Security, error handling, limits and upkeep points for this kind of integration were noted (Section 3.7).

Note: this is an exploration of the approach. The code is illustrative and was not deployed or run here. The endpoint, connection, form and field names are placeholders, and the syntax should be checked against the current Zoho documentation.

241401106

3. Output

3.1 Architecture of the pro-code extension



Zoho Creator keeps the form and the workflow. After a successful submission the workflow calls a Deluge custom function, which sends an HTTPS request through a connection to an Advanced I/O function on Zoho Catalyst. The Catalyst function runs the scoring logic, can use its own storage or an outside API if needed, and returns a JSON response that the workflow stores in the record.

3.2 Comparison of extension options

Option	*What it is*	*Skill needed*	*Best used for*
Workflow actions	Built-in actions such as send notification, add record or update record.	Low (no code)	Standard, simple automation.
Deluge custom function	A reusable standalone function written in Deluge and called from workflows.	Low to medium	Business rules reused inside Creator.
Creator Custom API	A REST endpoint exposed by the Creator application itself.	Medium	Letting other systems call into Creator.
Catalyst function	A serverless function in Node.js, Java or Python that runs outside Creator and is called over HTTPS.	Medium to high (pro-code)	Complex logic, libraries, heavy processing, outside systems.

241401106

3.3 Deluge custom function (illustrative)

```
// Standalone function: getPriorityLevel
string getPriorityLevel(int age, string conditions)
{
    payload = Map();
    payload.put("age", age);
    payload.put("conditions", conditions);

    response = invokeurl
    [
        url :"<Catalyst Advanced I/O endpoint>/priority"
        type :POST
        parameters :payload.toString()
        headers :{"Content-Type":"application/json"}
        connection :"catalyst_connection"
    ];
    return response.get("priority");
}

// Called from the Patient Appointment form workflow (on success)
level = thisapp.getPriorityLevel(input.Age, input.Existing_Conditions.toString());
Patient_Appointment[ID == input.ID].Priority_Level = level;
```

The custom function packs the request once, so any workflow can reuse it. The workflow passes the age and the existing conditions, receives the priority level and stores it with a criteria-based update on ID == input.ID. The URL, connection, form and field names are placeholders.

3.4 Catalyst Advanced I/O function in Node.js (illustrative)

```
const express = require("express");
const app = express();
app.use(express.json());

const CONDITION_WEIGHTS = { "Heart Disease": 3, "Diabetes":  , "Asthma": 1 };

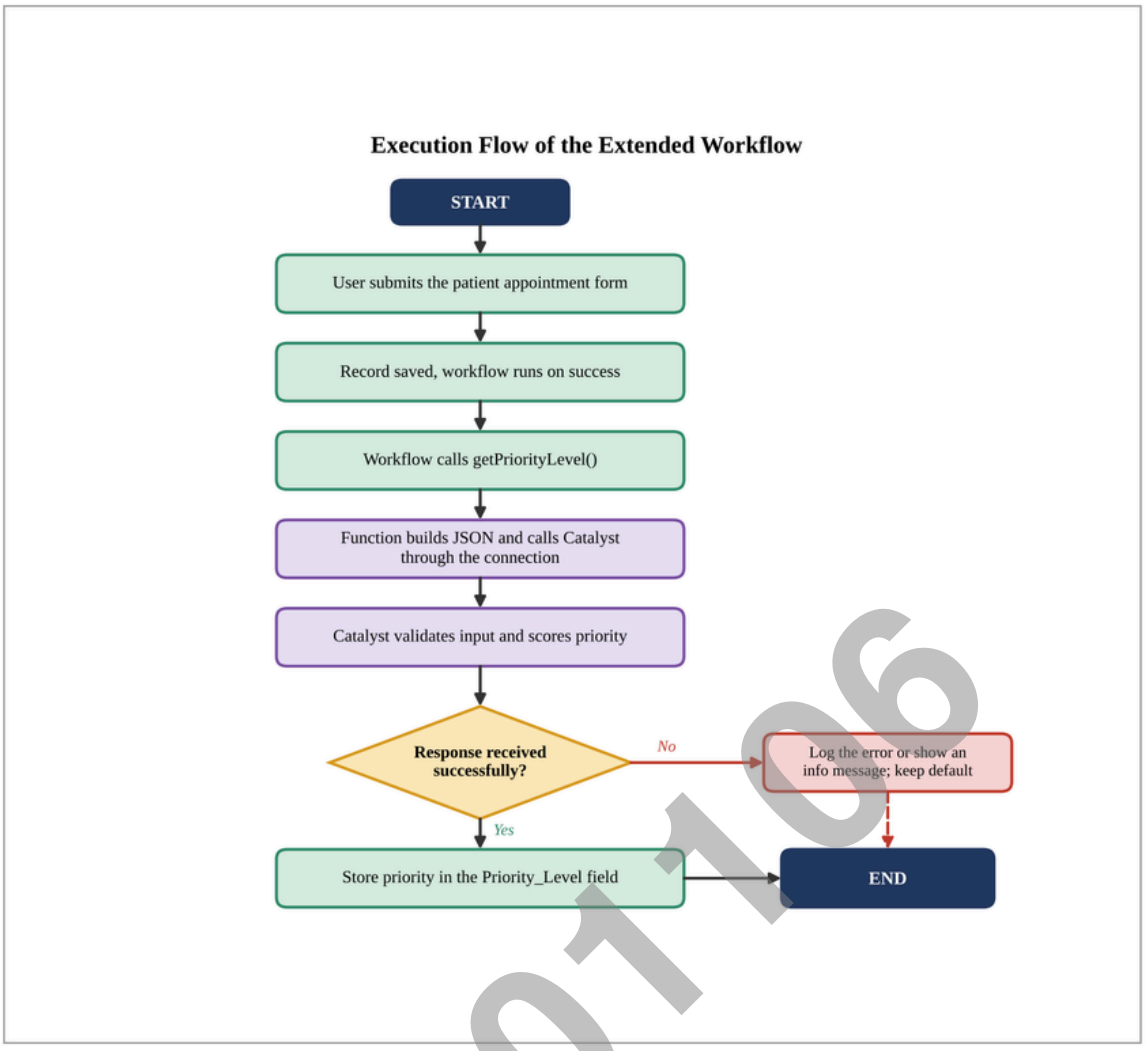
app.post("/priority", (req, res) => {
  const { age, conditions } = req.body;
  if (typeof age !== "number") {
    return res.status(400).json({ error: "age must be a number" });
  }
  let score = age >= 60 ? 2 : age < 5 ? 1 : 0;
  for (const name of (conditions || "").split(",")) {
    score += CONDITION_WEIGHTS[name.trim()] || 0;
  }
  const priority = score >= 4 ? "High" : score >= 2 ? "Medium" : "Normal";
  res.status(200).json({ priority });
});

module.exports = app;
```

The function reads the JSON body, rejects an age that is not a number, adds points for age and for each listed condition, and returns the priority. The weights and thresholds are only an example for demonstration; real clinical rules would have to be defined and approved by medical staff.

241401106

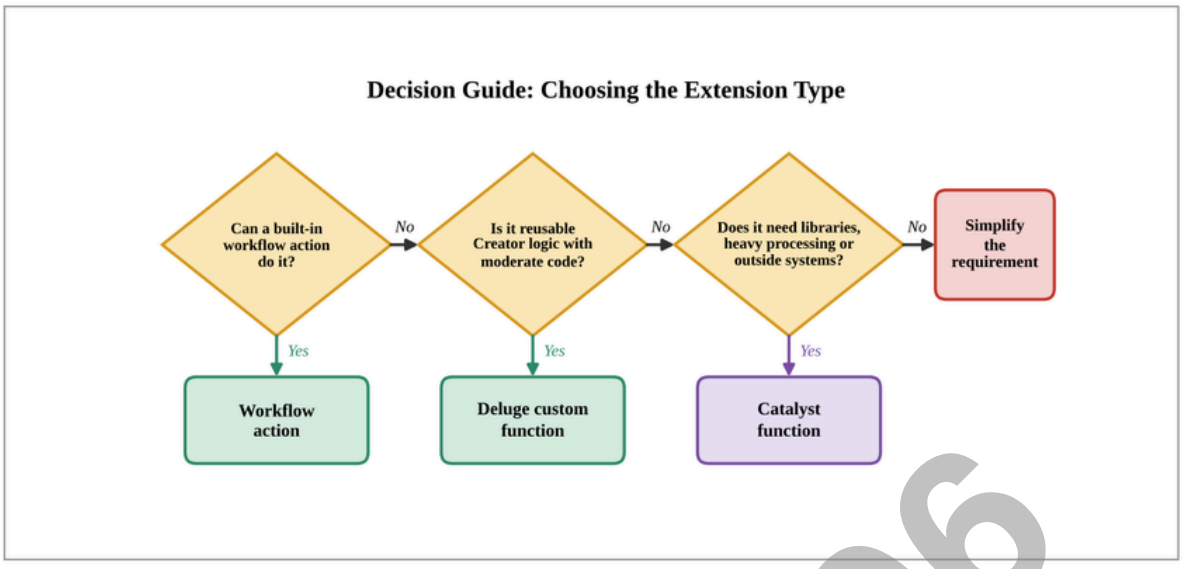
3.5 Execution flow of the extended workflow



The flowchart shows the path from form submission to record update. If a response is not received successfully, the error is logged or an information message is shown and the default value is kept, so the booking is never lost because of the extension.

241401106

3.6 Decision guide for choosing the extension type



The guide prefers built-in workflow actions first, then a Deluge custom function for reusable Creator logic, and uses a Catalyst function only when libraries, heavy processing or outside systems are needed. If none of these fits, the requirement should be simplified.

3.7 Benefits and precautions

Aspect	*Benefit*	*Precaution*
Flexibility	Any logic or library can be used in a language such as Node.js, Java or Python.	More code to write, test and maintain than low-code.
Reuse	A custom function is written once and called from many workflows.	Keep the inputs and outputs of the function stable.
Separation	Heavy work runs outside Creator, so the application stays light.	Adds a network call, so set timeouts and handle errors.
Security	Credentials stay in a connection, not in the script.	Keep keys out of code, restrict access and validate input.
Limits and cost	The platform handles serverless scaling.	Check current execution limits and pricing in the documentation.

4. Final Result

The exploration showed how a low-code Zoho Creator application can be extended in two steps: a reusable Deluge custom function inside Creator, and a pro-code Zoho Catalyst function reached over HTTPS through a connection. An architecture, illustrative code, an execution flow and a decision guide were prepared for an appointment priority scoring extension of a patient appointment scheduling system.

The comparison showed that workflow actions and custom functions cover most needs, and that Catalyst should be added only for complex logic, libraries or outside systems, with care for security, error handling and limits. The exercise met its objective of exploring pro-code extension. The code was not deployed or tested here, so results of a live run are not claimed.

241401106